
Oliver Krauss

Modern Code - Arrays Search & Sorting

ModernCode | MTD.ba VZ SS25

Hagenberg 18.11.2025

HAGENBERG | LINZ | STEYR | WELS



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

Why Learn Search and Sort Algorithms?

- **Algorithmic Thinking**: Learn to break down complex problems into systematic steps
- **Performance Awareness**: Understand that different solutions have different costs
- **Problem-Solving Skills**: Develop strategies for approaching new challenges
- **Code Quality**: Write efficient, well-designed solutions from the start
- **Foundation for Advanced Topics**: These patterns appear in databases, AI, and system design

The Two Fundamental Problems

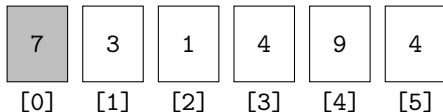
- **Search Problem:** "Is this item in my collection? Where is it?"
 - Linear search: Check every item (works on any data)
 - Binary search: Eliminate half each step (requires sorted data)
- **Sort Problem:** "Arrange my items in a specific order"
 - Simple sorts: Easy to understand, slower for large data
 - Efficient sorts: More complex, much faster for large data

Linear Search: The Simple Approach

- **Goal:** Find if a value exists in an array and where
- **Strategy:** Check every element one by one from start to finish
- **Works on:** Any array (sorted or unsorted)
- **Simple but complete:** Always finds the answer, but may need to check everything

Linear Search: Code

```
1  int[] a = {7, 3, 1, 4, 9, 4};
2  int target = 4;
3
4  // first or -1 if not found
5  int index = -1;
6  for (int i = 0; i < a.length;
       i++) {
7      if (a[i] == target) {
8          index = i;
9          break;
10     }
11 }
```

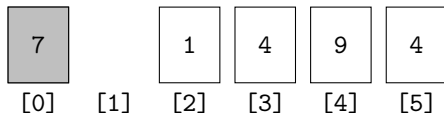


Step-by-step execution:

- i=0: Check `a[0]=7`, not equal to 4, continue

Linear Search: Code

```
1  int[] a = {7, 3, 1, 4, 9, 4};
2  int target = 4;
3
4  // first or -1 if not found
5  int index = -1;
6  for (int i = 0; i < a.length;
       i++) {
7      if (a[i] == target) {
8          index = i;
9          break;
10     }
11 }
```

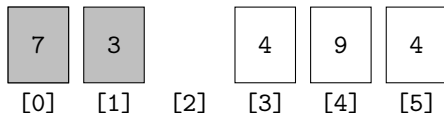


Step-by-step execution:

- i=0: Check $a[0]=7$, not equal to 4, continue
- i=1: Check $a[1]=3$, not equal to 4, continue

Linear Search: Code

```
1  int[] a = {7, 3, 1, 4, 9, 4};
2  int target = 4;
3
4  // first or -1 if not found
5  int index = -1;
6  for (int i = 0; i < a.length;
       i++) {
7      if (a[i] == target) {
8          index = i;
9          break;
10     }
11 }
```

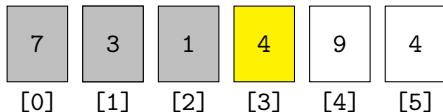


Step-by-step execution:

- i=0: Check $a[0]=7$, not equal to 4, continue
- i=1: Check $a[1]=3$, not equal to 4, continue
- i=2: Check $a[2]=1$, not equal to 4, continue

Linear Search: Code

```
1  int[] a = {7, 3, 1, 4, 9, 4};
2  int target = 4;
3
4  // first or -1 if not found
5  int index = -1;
6  for (int i = 0; i < a.length;
       i++) {
7      if (a[i] == target) {
8          index = i;
9          break;
10     }
11 }
```

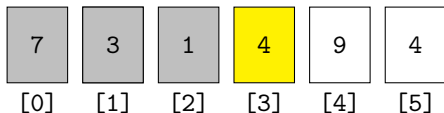


Step-by-step execution:

- i=0: Check **a[0]=7**, not equal to 4, continue
- i=1: Check **a[1]=3**, not equal to 4, continue
- i=2: Check **a[2]=1**, not equal to 4, continue
- i=3: Check **a[3]=4**, equal to 4! Set **index=3** and break

Linear Search: Code

```
1  int[] a = {7, 3, 1, 4, 9, 4};
2  int target = 4;
3
4  // first or -1 if not found
5  int index = -1;
6  for (int i = 0; i < a.length;
       i++) {
7      if (a[i] == target) {
8          index = i;
9          break;
10     }
11 }
```

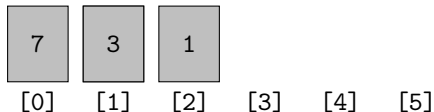


Step-by-step execution:

- **i=0**: Check **a[0]=7**, not equal to 4, continue
- **i=1**: Check **a[1]=3**, not equal to 4, continue
- **i=2**: Check **a[2]=1**, not equal to 4, continue
- **i=3**: Check **a[3]=4**, equal to 4! Set **index=3** and break
- Result: **index=3** (first occurrence of 4)

Linear Search: Code

```
1  int[] a = {7, 3, 1, 4, 9, 4};
2  int target = 4;
3
4  // first or -1 if not found
5  int index = -1;
6  for (int i = 0; i < a.length;
       i++) {
7      if (a[i] == target) {
8          index = i;
9          break;
10     }
11 }
```

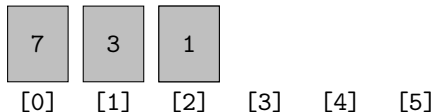


Step-by-step execution:

- **i=0**: Check **a[0]=7**, not equal to 4, continue
- **i=1**: Check **a[1]=3**, not equal to 4, continue
- **i=2**: Check **a[2]=1**, not equal to 4, continue
- **i=3**: Check **a[3]=4**, equal to 4! Set **index=3** and break
- Result: **index=3** (first occurrence of 4)

Linear Search: Code

```
1  int[] a = {7, 3, 1, 4, 9, 4};
2  int target = 4;
3
4  // first or -1 if not found
5  int index = -1;
6  for (int i = 0; i < a.length;
       i++) {
7      if (a[i] == target) {
8          index = i;
9          break;
10     }
11 }
```

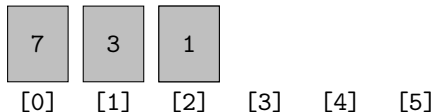


Step-by-step execution:

- **i=0**: Check **a[0]=7**, not equal to 4, continue
- **i=1**: Check **a[1]=3**, not equal to 4, continue
- **i=2**: Check **a[2]=1**, not equal to 4, continue
- **i=3**: Check **a[3]=4**, equal to 4! Set **index=3** and break
- Result: **index=3** (first occurrence of 4)

Linear Search: Code

```
1  int[] a = {7, 3, 1, 4, 9, 4};
2  int target = 4;
3
4  // first or -1 if not found
5  int index = -1;
6  for (int i = 0; i < a.length;
       i++) {
7      if (a[i] == target) {
8          index = i;
9          break;
10     }
11 }
```



Step-by-step execution:

- **i=0**: Check **a[0]=7**, not equal to 4, continue
- **i=1**: Check **a[1]=3**, not equal to 4, continue
- **i=2**: Check **a[2]=1**, not equal to 4, continue
- **i=3**: Check **a[3]=4**, equal to 4! Set **index=3** and break
- Result: **index=3** (first occurrence of 4)

Binary Search: A Faster Alternative

- **Problem with Linear Search:** Must check every element in worst case
- **Binary Search Strategy:** Eliminate half the possibilities each step
- **Requirement:** Array must be sorted
- **Trade-off:** More complex code, but much faster for large arrays

Binary Search: How It Works

- **Step 1:** Look at the middle element
- **Step 2:** If it's our target, we're done!
- **Step 3:** If target is smaller, search left half
- **Step 4:** If target is larger, search right half
- **Step 5:** Repeat until found or search space is empty

Why it's fast:

- Each step eliminates half the remaining possibilities
- For 1000 elements: max 10 steps (instead of 1000)
- For 1,000,000 elements: max 20 steps (instead of 1,000,000)

Binary Search: Setup and Variables

```
1  int[] sorted = {2, 4, 6, 8, 10, 12, 14};
2  int target = 8;
3
4  int left = 0, right = sorted.length - 1;
5  int index = -1;
```

Key variables:

- **left**: Start of search range (initially 0)
- **right**: End of search range (initially last index)
- **index**: Where we found the target (initially -1 for "not found")
- **target**: The value we're looking for

Binary Search: The Main Loop

```
1  while (left <= right) {
2      int mid = (left + right) / 2;
3      if (sorted[mid] == target) {
4          index = mid;
5          break;
6      } else if (sorted[mid] < target) {
7          left = mid + 1;
8      } else {
9          right = mid - 1;
10     }
11 }
```

What happens in each iteration:

- Calculate middle position: `mid = (left + right) / 2`
- Check if middle element is our target
- If not, eliminate half the search space

Binary Search: Step-by-Step Execution

Step-by-step execution:

```
1  int[] sorted = {2, 4, 6, 8,  
    10, 12, 14};  
2  int target = 10;  
3  int left = 0, right = 6;
```

-- **Step 1:** left=0, right=6, mid=3

2	4	6	8	10	12	14
[0]	[1]	[2]	[3]	[4]	[5]	[6]

Binary Search: Step-by-Step Execution

```
1  int[] sorted = {2, 4, 6, 8,  
    10, 12, 14};  
2  int target = 10;  
3  int left = 0, right = 6;
```

Step-by-step execution:

- **Step 1:** left=0, right=6, mid=3
- Check `sorted[3]=8`. Since $8 < 10$, target is in right half

2	4	6	8	10	12	14
[0]	[1]	[2]	[3]	[4]	[5]	[6]

Binary Search: Step-by-Step Execution

```
1  int[] sorted = {2, 4, 6, 8,  
    10, 12, 14};  
2  int target = 10;  
3  int left = 0, right = 6;
```

Step-by-step execution:

- **Step 1:** left=0, right=6, mid=3
- Check `sorted[3]=8`. Since $8 < 10$, target is in right half
- Update: `left = mid + 1 = 4`

2	4	6	8	10	12	14
[0]	[1]	[2]	[3]	[4]	[5]	[6]

Binary Search: Step-by-Step Execution

```
1  int[] sorted = {2, 4, 6, 8,  
    10, 12, 14};  
2  int target = 10;  
3  int left = 0, right = 6;
```

Step-by-step execution:

- **Step 1:** left=0, right=6, mid=3
- Check `sorted[3]=8`. Since $8 < 10$, target is in right half
- Update: `left = mid + 1 = 4`
- **Step 2:** left=4, right=6, mid=5

2	4	6	8	10	12	14
[0]	[1]	[2]	[3]	[4]	[5]	[6]

Binary Search: Step-by-Step Execution

```
1  int[] sorted = {2, 4, 6, 8,  
    10, 12, 14};  
2  int target = 10;  
3  int left = 0, right = 6;
```

2	4	6	8	10	12	14
[0]	[1]	[2]	[3]	[4]	[5]	[6]

Step-by-step execution:

- **Step 1:** left=0, right=6, mid=3
- Check `sorted[3]=8`. Since $8 < 10$, target is in right half
- Update: `left = mid + 1 = 4`
- **Step 2:** left=4, right=6, mid=5
- Check `sorted[5]=12`. Since $12 > 10$, target is in left half

Binary Search: Step-by-Step Execution

```
1  int[] sorted = {2, 4, 6, 8,  
    10, 12, 14};  
2  int target = 10;  
3  int left = 0, right = 6;
```

2	4	6	8	10	12	14
[0]	[1]	[2]	[3]	[4]	[5]	[6]

Step-by-step execution:

- **Step 1:** left=0, right=6, mid=3
- Check `sorted[3]=8`. Since $8 < 10$, target is in right half
- Update: `left = mid + 1 = 4`
- **Step 2:** left=4, right=6, mid=5
- Check `sorted[5]=12`. Since $12 > 10$, target is in left half
- Update: `right = mid - 1 = 4`

Binary Search: Step-by-Step Execution

```
1  int[] sorted = {2, 4, 6, 8,  
    10, 12, 14};  
2  int target = 10;  
3  int left = 0, right = 6;
```

2	4	6	8	10	12	14
[0]	[1]	[2]	[3]	[4]	[5]	[6]

Step-by-step execution:

- **Step 1:** left=0, right=6, mid=3
- Check `sorted[3]=8`. Since $8 < 10$, target is in right half
- Update: `left = mid + 1 = 4`
- **Step 2:** left=4, right=6, mid=5
- Check `sorted[5]=12`. Since $12 > 10$, target is in left half
- Update: `right = mid - 1 = 4`
- **Step 3:** left=4, right=4, mid=4

Binary Search: Step-by-Step Execution

```
1  int[] sorted = {2, 4, 6, 8,  
    10, 12, 14};  
2  int target = 10;  
3  int left = 0, right = 6;
```

2	4	6	8	10	12	14
[0]	[1]	[2]	[3]	[4]	[5]	[6]

Step-by-step execution:

- **Step 1:** left=0, right=6, mid=3
- Check `sorted[3]=8`. Since $8 < 10$, target is in right half
- Update: `left = mid + 1 = 4`
- **Step 2:** left=4, right=6, mid=5
- Check `sorted[5]=12`. Since $12 > 10$, target is in left half
- Update: `right = mid - 1 = 4`
- **Step 3:** left=4, right=4, mid=4
- Check `sorted[4]=10`. Match! Found at index 4

Search Algorithm Comparison

- **Linear Search:** Works on any array, checks every element
- **Binary Search:** Only works on sorted arrays, eliminates half each step
- **When to use:** Linear for unsorted, binary for sorted data
- **Trade-off:** Binary is faster but requires sorted data

Sorting Strategy Overview

How do we sort?

- **Compare:** "Is A bigger than B?"
- **Swap:** "Move A to the right position"
- **Repeat:** Until everything is in order

Key Questions:

- How many comparisons?
- How many swaps?
- How much extra memory?

Example: Sorting [3, 1, 4, 2]

1. Compare 3 and 1 → swap → [1, 3, 4, 2]
2. Compare 3 and 4 → no swap
3. Compare 4 and 2 → swap → [1, 3, 2, 4]
4. Compare 3 and 2 → swap → [1, 2, 3, 4]

Sorting Algorithm Categories

Simple sorts (easy to understand and implement)

- **Bubble Sort:** Compare neighbors, bubble up
- **Selection Sort:** Find minimum, put in front
- **Insertion Sort:** Insert each element in right place
- **Shaker Sort:** Bidirectional bubble sort

Advanced sorts (better performance for large arrays)

- **Merge Sort:** Divide array, sort halves, merge
- **Quick Sort:** Pick pivot, partition, sort recursively
- **Shell Sort:** Sort with decreasing gaps
- **Heap Sort:** Build heap, extract maximums

Key Sorting Properties

Memory Usage

- **In-place**: Sort within the original array (uses 1 extra space)
- **Out-of-place**: Create new array for sorted result (uses n extra spaces)

Stability

- **Stable**: Preserves relative order of equal elements
- **Unstable**: May change relative order of equal elements

Adaptive

- **Adaptive**: Performance improves on partially sorted data
- **Non-adaptive**: Performance is the same regardless of input order

Algorithm Properties Comparison

Memory Usage

- **In-place**: Selection, Bubble, Insertion, Shaker, Quick, Heap, Shell
- **Out-of-place**: Merge Sort

Stability

- **Stable**: Bubble, Insertion, Merge, Shaker
- **Unstable**: Selection, Quick, Heap, Shell

Adaptive

- **Yes**: Bubble, Insertion, Shaker
- **No**: Selection, Quick, Merge, Heap, Shell

Simple Sorting Algorithms: Introduction

- **Goal:** Learn fundamental sorting strategies that are easy to understand
- **Approach:** Build intuition with simple, visual algorithms
- **Pattern:** Compare elements and swap them into correct positions
- **Key insight:** Each algorithm has a different strategy for finding the right position

We'll cover (if time permits):

- Bubble Sort: Compare and swap neighbors
- Selection Sort: Find minimum and place it
- Insertion Sort: Insert each element in its place
- Shaker Sort: Bidirectional bubble sort

The Swap Operation

What is swapping?

- Exchange two elements in an array
- Move element A to position B
- Move element B to position A

Swap method:

```
1  public static void swap(int[] arr, int i, int j) {  
2      int temp = arr[i];  
3      arr[i] = arr[j];  
4      arr[j] = temp;  
5  }
```

Example: Swapping elements 2 and 4

1. Original: [3, 2, 7, 4, 1]
2. After swap: [3, 4, 7, 2, 1]
3. Elements 2 and 4 changed places

Usage: `swap(arr, 1, 3);` // Swaps elements at positions 1 and 3

Bubble Sort: Core Concept

Strategy: Compare and swap adjacent elements

- **Key idea:** "Bubble" the largest element to the end in each pass
- **Process:** Compare each pair of adjacent elements, swap if out of order
- **Result:** After each pass, the largest unsorted element reaches its final position
- **Optimization:** Stop early if no swaps occur (array is sorted)

Bubble Sort: Step-by-Step

```
1  // Pass 1: Compare adjacent elements
2  int[] arr = {64, 34, 25, 12, 22, 11, 90};
3  for (int i = 0; i < arr.length - 1; i++) {
4      if (arr[i] > arr[i + 1]) {
5          swap(arr, i, i + 1);
6      }
7  }
8  // After pass 1: [34, 25, 12, 22, 11, 64, 90]
9  // Largest element "bubbles" to the end
```

Bubble Sort: Complete Implementation

```
1  void bubbleSort(int[] a) {
2      boolean swapped;
3      for (int pass = 0; pass < a.length - 1; pass++) {
4          swapped = false;
5          for (int i = 0; i < a.length - 1 - pass; i++) {
6              if (a[i] > a[i + 1]) {
7                  swap(a, i, i + 1);
8                  swapped = true;
9              }
10         }
11         if (!swapped) break; // already sorted
12     }
13 }
```

Selection Sort: Core Concept

Strategy: Find the minimum element and place it at the beginning

- **Key idea:** In each iteration, find the smallest unsorted element
- **Process:** Scan the unsorted portion, identify minimum, swap to correct position
- **Result:** After each iteration, one more element is in its final position
- **Invariant:** Elements before current position are sorted

Selection Sort: Step-by-Step

```
1  // Step 1: Find the smallest element
2  int[] arr = {64, 25, 12, 22, 11};
3  int minIndex = 0;
4  for (int j = 1; j < arr.length; j++) {
5      if (arr[j] < arr[minIndex]) minIndex = j;
6  }
7  // minIndex = 4 (value 11)
8
9  // Step 2: Swap with first position
10 swap(arr, 0, minIndex);
11 // Array: [11, 25, 12, 22, 64]
```

Selection Sort: Complete Implementation

```
1  void selectionSort(int[] a) {  
2      for (int i = 0; i < a.length - 1; i++) {  
3          int minIndex = i;  
4          for (int j = i + 1; j < a.length; j++) {  
5              if (a[j] < a[minIndex]) minIndex = j;  
6          }  
7          if (minIndex != i) {  
8              swap(a, i, minIndex);  
9          }  
10     }  
11 }
```

Insertion Sort: Core Concept

Strategy: Insert each element into its correct position in the sorted portion

- **Key idea:** Build the sorted array one element at a time
- **Process:** Take next element, shift larger elements right, insert in correct spot
- **Result:** After each insertion, the left portion remains sorted
- **Invariant:** Elements to the left of current position are sorted

Insertion Sort: Step-by-Step

```
1  // Insert element at position 1
2  int[] arr = {12, 11, 13, 5, 6};
3  int key = arr[1]; // 11
4  int j = 0;
5
6  // Shift elements greater than key
7  while (j >= 0 && arr[j] > key) {
8      arr[j + 1] = arr[j];
9      j--;
10 }
11 arr[j + 1] = key;
12 // After insertion: [11, 12, 13, 5, 6]
```

Insertion Sort: Complete Implementation

```
1  void insertionSort(int[] a) {  
2      for (int i = 1; i < a.length; i++) {  
3          int key = a[i];  
4          int j = i - 1;  
5          while (j >= 0 && a[j] > key) {  
6              a[j + 1] = a[j];  
7              j--;  
8          }  
9          a[j + 1] = key;  
10     }  
11 }
```

Shaker Sort: Core Concept

Strategy: Bidirectional bubble sort - bubble in both directions

- **Key idea:** Alternate between forward and backward passes
- **Process:** Forward pass bubbles largest to end, backward pass bubbles smallest to beginning
- **Result:** Both ends get sorted simultaneously, reducing total passes needed
- **Optimization:** Stop when no swaps occur in either direction

Concept: Forward: [64, 34, 25, 12, 22, 11] → [34, 25, 12, 22, 11, 64]

Backward: [34, 25, 12, 22, 11, 64] → [11, 34, 25, 12, 22, 64]

Shaker Sort: Forward Pass

```
1  void shakerSort(int[] a) {
2      boolean swapped = true;
3      int start = 0, end = a.length - 1;
4      while (swapped) {
5          swapped = false;
6          // Forward pass
7          for (int i = start; i < end; i++) {
8              if (a[i] > a[i + 1]) {
9                  swap(a, i, i + 1);
10                 swapped = true;
11             }
12         }
13         end--;
```

Shaker Sort: Backward Pass

```
14      // Backward pass
15      for (int i = end; i > start; i--) {
16          if (a[i] < a[i - 1]) {
17              swap(a, i, i - 1);
18              swapped = true;
19          }
20      }
21      start++;
22  }
23 }
```

Efficient Sorting: Introduction

- **Goal:** Learn advanced sorting strategies that are much faster for large arrays
- **Approach:** Use divide-and-conquer and other sophisticated techniques
- **Pattern:** Break problem into smaller parts, solve recursively, combine results
- **Key insight:** Each algorithm uses a different strategy for efficient sorting

We'll cover (if time permits):

- **Merge Sort:** Divide array, sort halves, merge
- **Quick Sort:** Pick pivot, partition, sort recursively
- **Shell Sort:** Sort with decreasing gaps
- **Heap Sort:** Build heap, extract maximums

Merge Sort: Core Concept

Strategy: Divide array in half, sort each half, merge results

- **Key idea:** Split problem into smaller identical problems
- **Process:** Recursively sort left and right halves, then merge them
- **Result:** After merging, the combined array is sorted
- **Stable:** Preserves relative order of equal elements

Merge Sort: Step-by-Step

```
1   Example: Sorting [38, 27, 43, 3, 9, 82, 10]
2   // Step 1: Divide into halves
3   Left: [38, 27, 43, 3]    Right: [9, 82, 10]
4
5   // Step 2: Sort left half recursively
6   [38, 27, 43, 3] -> [3, 27, 38, 43]
7
8   // Step 3: Sort right half recursively
9   [9, 82, 10] -> [9, 10, 82]
10
11  // Step 4: Merge sorted halves
12  [3, 27, 38, 43] + [9, 10, 82] -> [3, 9, 10, 27, 38, 43, 82]
```

Merge Sort: Main Function

```
1 void mergeSort(int[] arr, int left, int right) {  
2     if (left < right) {  
3         int mid = (left + right) / 2;  
4         mergeSort(arr, left, mid);  
5         mergeSort(arr, mid + 1, right);  
6         merge(arr, left, mid, right);  
7     }  
8 }
```

- **Base case:** If $\text{left} \geq \text{right}$, array has 0 or 1 elements (already sorted)
- **Divide:** Split array into two halves at middle position
- **Conquer:** Recursively sort both halves
- **Combine:** Merge the sorted halves back together

Merge Function: How It Works

Goal: Combine two sorted arrays into one sorted array

The merge process:

1. **Create temporary arrays** to hold the two sorted halves
2. **Copy data** from original array to temp arrays
3. **Compare and merge** elements from both temp arrays
4. **Handle remaining** elements from either array

Why temp arrays?

- Allows us to merge back into the original array safely
- Prevents overwriting data we still need to read
- Makes the algorithm easier to understand and implement

Merge Sort: Setup Phase

```
1  void merge(int[] arr, int left, int mid, int right) {  
2      // Create temporary arrays  
3      int[] leftArr = new int[mid - left + 1];  
4      int[] rightArr = new int[right - mid];  
5  
6      // Copy data to temp arrays  
7      for (int i = 0; i < leftArr.length; i++) {  
8          leftArr[i] = arr[left + i];  
9      }  
10     for (int j = 0; j < rightArr.length; j++) {  
11         rightArr[j] = arr[mid + 1 + j];  
12     }
```

Merge Sort: Merge Phase

```
13      // Merge the temp arrays back
14      int i = 0, j = 0, k = left;
15      while (i < leftArr.length && j < rightArr.length) {
16          if (leftArr[i] <= rightArr[j]) {
17              arr[k] = leftArr[i];
18              i++;
19          } else {
20              arr[k] = rightArr[j];
21              j++;
22          }
23          k++;
24      }
```

Merge Sort: Copy Elements

```
25      // Copy remaining elements
26      while (i < leftArr.length) arr[k++] = leftArr[i++];
27      while (j < rightArr.length) arr[k++] = rightArr[j++];
28  }
```

Quick Sort: Core Concept

Strategy: Pick a pivot element, partition around it, sort partitions recursively

- **Key idea:** Choose a "pivot" and arrange elements around it
- **Process:** Partition array so all elements $<$ pivot are left, $>$ pivot are right
- **Result:** Pivot is in final position, sort left and right partitions
- **In-place:** Sorts array without additional memory

Quick Sort: Step-by-Step

```
1  Example: Sorting [64, 25, 12, 22, 11, 90]
2  // Step 1: Choose pivot (last element: 90)
3  // Step 2: Partition around pivot
4  [64, 25, 12, 22, 11] | 90
5  // All elements < 90 stay left, > 90 go right
6
7  // Step 3: Recursively sort partitions
8  Left: [64, 25, 12, 22, 11] -> [11, 12, 22, 25, 64]
9  Right: [90] (already sorted)
10
11 // Step 4: Combine: [11, 12, 22, 25, 64, 90]
```

Quick Sort: How It Works

The Quick Sort Process:

1. **Base case:** If $low \geq high$, array has 0 or 1 elements (already sorted)
2. **Partition:** Rearrange array so pivot is in correct position
3. **Recurse:** Sort left and right partitions separately
4. **Result:** Pivot is in final position, partitions are sorted

Key insight: Each partition step puts one element (the pivot) in its final sorted position, then we recursively sort the remaining elements.

Quick Sort: Main Function

```
1 void quickSort(int[] arr, int low, int high) {  
2     if (low < high) {  
3         int pivotIndex = partition(arr, low, high);  
4         quickSort(arr, low, pivotIndex - 1);  
5         quickSort(arr, pivotIndex + 1, high);  
6     }  
7 }
```

Quick Sort: Partition Process

How Partition Works:

1. **Choose pivot:** Use last element as pivot
2. **Track position:** i marks end of "smaller than pivot" section
3. **Move elements:** Swap smaller elements to left of pivot position
4. **Place pivot:** Put pivot in its final sorted position

Key insight: After partitioning, the pivot is in its final sorted position, and all elements to its left are smaller, all to its right are larger.

Quick Sort: Partition Function

```
1  int partition(int[] arr, int low, int high) {
2      int pivot = arr[high];
3      int i = low - 1;
4
5      for (int j = low; j < high; j++) {
6          if (arr[j] <= pivot) {
7              i++;
8              swap(arr, i, j);
9          }
10     }
11     swap(arr, i + 1, high);
12     return i + 1;
13 }
```

Shell Sort: Core Concept

Strategy: Sort with decreasing gaps between compared elements

- **Key idea:** Start with large gaps, reduce gap size until gap = 1
- **Process:** Use insertion sort with different gap sizes
- **Result:** Early passes move elements closer to final positions
- **In-place:** Sorts array without additional memory

Shell Sort: Step-by-Step

```
1  Example: Sorting [64, 34, 25, 12, 22, 11, 90, 5]
2  // Gap sequence: 4, 2, 1
3  // Gap 4: Compare elements 4 positions apart
4  [64, 34, 25, 12, 22, 11, 90, 5]
5  ^       ^       ^       ^
6  //  Compare and swap if needed
7  // Gap 2: Compare elements 2 positions apart
8  [22, 11, 25, 5, 34, 12, 90, 64]
9  ^     ^     ^     ^     ^     ^
10 //  Compare and swap if needed
11 // Gap 1: Standard insertion sort
12 Final sorted array: [5, 11, 12, 22, 25, 34, 64, 90]
```

Shell Sort: How It Works

The Shell Sort Process:

1. **Start with large gap:** Begin with $\text{gap} = n/2$
2. **Insertion sort with gap:** Sort elements that are gap positions apart
3. **Reduce gap:** Divide gap by 2 each iteration
4. **Repeat until gap = 1:** Final pass is standard insertion sort

Early passes with large gaps move elements much closer to their final positions, making the final insertion sort very efficient.

Shell Sort: Complete Implementation

```
1 void shellSort(int[] arr) {  
2     int n = arr.length;  
3     // Start with large gap, reduce by half each time  
4     for (int gap = n / 2; gap > 0; gap /= 2) {  
5         for (int i = gap; i < n; i++) { // insertion sort  
6             int temp = arr[i];  
7             int j;  
8             for (j = i; j >= gap && arr[j - gap] > temp; j -= gap) {  
9                 arr[j] = arr[j - gap];  
10            }  
11            arr[j] = temp;  
12        }  
13    }
```

Heap Sort: Core Concept

Strategy: Build a heap, repeatedly extract the maximum element

- **Key idea:** Use heap data structure to efficiently find maximum
- **Process:** Build max-heap, swap root with last element, heapify
- **Result:** After each extraction, largest element goes to end
- **In-place:** Sorts array without additional memory

Heap Sort: Step-by-Step

```
1  // Example: Sorting [64, 34, 25, 12, 22, 11, 90]
2  // Step 1: Build max-heap
3  //      90
4  //    /  \
5  //   34   25
6  //  / \   / \
7  // 12 22 11 64
8
9  // Step 2: Extract max (90), swap with last element
10 // [64, 34, 25, 12, 22, 11, 90] -> [64, 34, 25, 12, 22, 11, 90]
11
12 // Step 3: Heapify remaining elements, repeat
13 // Continue until all elements are in sorted positions
```

Heap Sort: How It Works

The Heap Sort Process:

1. **Build heap:** Convert array into max-heap structure
2. **Extract max:** Swap root with last element, reduce heap size
3. **Maintain heap:** Re-heapify after each extraction
4. **Result:** Array becomes sorted in ascending order

We repeatedly extract the maximum element (root of heap) and place it at the end of the array, building the sorted portion from right to left.

Heap Sort: Main Function

```
1  void heapSort(int[] arr) {
2      int n = arr.length;
3
4      // Build max heap
5      for (int i = n / 2 - 1; i >= 0; i--) {
6          heapify(arr, n, i);
7      }
8      // Extract elements from heap one by one
9      for (int i = n - 1; i > 0; i--) {
10         swap(arr, 0, i);
11         heapify(arr, i, 0);
12     }
13 }
```

Heap Sort: Heapify Process

How Heapify Works:

1. **Find largest**: Compare node with its children
2. **Swap if needed**: Move larger child up if it's bigger than parent
3. **Recurse**: Continue heapifying down the tree
4. **Result**: Maintains max-heap property

Heapify ensures that a node and its children satisfy the max-heap property (parent $\geq$ children). If not, it swaps and continues down the tree.

Heap Sort: Heapify Function

```
1 void heapify(int[] arr, int n, int i) {  
2     int largest = i;  
3     int left = 2 * i + 1;  
4     int right = 2 * i + 2;  
5     if (left < n && arr[left] > arr[largest]) {  
6         largest = left;  
7     }  
8     if (right < n && arr[right] > arr[largest]) {  
9         largest = right;  
10    }  
11    if (largest != i) {  
12        swap(arr, i, largest);  
13        heapify(arr, n, largest);  
    }
```

Sorting Algorithm Comparison

Algorithm	Best	Average	Worst
Selection Sort	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$
Bubble Sort	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$
Insertion Sort	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$
Shaker Sort	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$
Shell Sort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n^{1.5})$	$\mathcal{O}(n^2)$
Merge Sort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$
Quick Sort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n^2)$
Heap Sort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$

Note: We'll learn about Big O notation in detail later. For now, focus on the relative performance.

Reality Check: Will You Implement Sorting?

In most cases, you **DON'T** implement sorting algorithms!

- **Use library functions:** `Arrays.sort()`, `Collections.sort()`
- **Highly optimized:** Libraries use the best algorithms for your data
- **Well-tested:** Battle-tested code with edge cases handled
- **Maintained:** Regular updates and performance improvements

When you **MIGHT** implement custom sorting:

- **Core library development:** Building programming languages or frameworks
- **Specialized data structures:** Custom containers with specific requirements
- **Performance-critical systems:** Real-time systems, embedded programming
- **Interview preparation:** Understanding algorithmic thinking

When to Use Which Algorithm

If you ever need to choose (rare!):

- **Selection Sort:** Simple, in-place, when memory is limited
- **Bubble Sort:** Educational purposes, very small arrays
- **Insertion Sort:** Small arrays, nearly sorted data
- **Merge Sort:** Stable sorting needed, consistent performance
- **Quick Sort:** General purpose, good average performance

The real value: Understanding these algorithms teaches you problem-solving patterns that apply to many other programming challenges!

Interactive Sorting Visualizations

Explore Sorting Algorithms Interactively

VisuAlgo - Sorting Visualizations

<https://visualgo.net/en/sorting>

OI Wiki - Sorting Introduction

<https://en.oj-wiki.org/basic/sort-intro/>

Recommended Learning Approach

- Use **VisuAlgo** to understand how algorithms work visually
- Use **OI Wiki** to dive deeper into the mathematical theory
- Practice implementing algorithms after understanding the concepts

Common Mistakes: Array Bounds

Array Index Errors:

- **Off-by-one errors:** Using `<=` instead of `<` in loop conditions
- **Array bounds:** Accessing `arr[arr.length]` instead of `arr[arr.length-1]`
- **Negative indices:** Not checking if index is valid before accessing
- **Empty arrays:** Not handling arrays with length 0

Example:

- `for (int i = 0; i <= arr.length; i++) // Wrong!`
- `for (int i = 0; i < arr.length; i++) // Correct`

Common Mistakes: Swapping and Sorting

Swapping Errors:

- **Wrong indices:** Swapping `arr[i]` with `arr[j]` when you meant `arr[i+1]`
- **Missing temp variable:** Trying to swap without temporary storage
- **Swapping same element:** `swap(arr, i, i)` does nothing

Sorting Logic Errors:

- **Wrong comparison:** Using `<` instead of `>` for ascending order
- **Missing base case:** Infinite recursion in recursive algorithms
- **Incorrect pivot choice:** Choosing wrong element as pivot in Quick Sort

Common Mistakes: Search Algorithms

Binary Search Pitfalls:

- **Unsorted data:** Using binary search on unsorted arrays
- **Integer overflow:** $(\text{left} + \text{right}) / 2$ can overflow
- **Wrong bounds update:** Not updating **left** or **right** correctly
- **Missing elements:** Not handling "not found" case properly

Linear Search Issues:

- **Not breaking early:** Continuing search after finding the element
- **Wrong return value:** Returning index instead of boolean (or vice versa)

Common Mistakes: Memory and Performance

Memory Issues:

- **Memory leaks:** Not cleaning up temporary arrays in Merge Sort
- **Stack overflow:** Deep recursion without proper base cases
- **Unnecessary copying:** Creating arrays when in-place sorting is possible

Performance Mistakes:

- **Using wrong algorithm:** Bubble Sort for large datasets
- **Inefficient comparisons:** Comparing strings with `==` instead of `.equals()`
- **Redundant operations:** Sorting already sorted data

How to Avoid These Mistakes

Best Practices:

- **Test with edge cases:** Empty arrays, single elements, already sorted
- **Use meaningful variable names:** `left`, `right`, `mid` instead of `i`, `j`, `k`
- **Add bounds checking:** Validate indices before array access
- **Use library functions:** `Arrays.sort()` when possible

Debugging Tips:

- **Print intermediate values:** See what's happening in each step
- **Use small test cases:** Easier to trace through manually
- **Check invariants:** Ensure your algorithm maintains expected properties

Summary: Arrays Sorting

Search Algorithms

- Linear search (any array)
- Binary search (sorted arrays)
- Trade-offs: speed vs requirements

Sorting Strategies

- Comparison-based sorting
- In-place vs out-of-place
- Stable vs unstable

Simple Sorts

- Selection, Bubble, Insertion, Shaker
- Easy to understand and implement
- Good for small arrays

Efficient Sorts

- Merge Sort (stable, consistent)
- Quick Sort (in-place, fast average)
- Shell Sort (gap-based improvement)
- Heap Sort (heap data structure)

Practical Reality

- Use library functions when possible
- **`Arrays.sort()`** is highly optimized
- Custom implementation for special cases

Key Insights

- Different algorithms for different needs
- Step-by-step implementation helps understanding
- Practice with small examples first